

All You Need to Know for LeetCode (Python)

This guide is a compact Python reference for solving LeetCode/NeetCode problems. Each mini-topic includes a short explanation and a tiny example with outputs.

1) Input and Output Patterns

Reading input is rare on LeetCode, but you should know how to parse strings when needed. Use `input()` for one line and `sys.stdin` for speed with large inputs. In LeetCode, you usually return values instead of printing them.

```
# Example: parse two ints from one line
line = "3 5"
a, b = map(int, line.split())
print(a + b)
```

Output:

```
8
```

2) Lists (Arrays)

Lists are the default array structure for most problems. They support $O(1)$ indexing, $O(1)$ append, and $O(n)$ insert/delete in the middle. Use slicing for subarrays, and be careful with copying vs referencing.

```
nums = [1, 2, 3]
nums.append(4)
print(nums)
print(nums[1:3])
```

Output:

```
[1, 2, 3, 4]
[2, 3]
```

3) Strings

Strings are immutable, so operations create new strings. Learn slicing, concatenation, and `.join()` for efficient building. LeetCode uses lots of string parsing and transformations.

```
s = "abc"
print(s[1:])
print("-".join(["a", "b", "c"]))
```

Output:

```
bc
a-b-c
```

4) Dictionaries (Hash Maps)

Dictionaries give average $O(1)$ insert, lookup, and delete. They are essential for frequency counting and fast lookups. Use `get` or `defaultdict` to simplify counting.

```
freq = {}
for ch in "aab":
    freq[ch] = freq.get(ch, 0) + 1
print(freq)
```

Output:

```
{'a': 2, 'b': 1}
```

5) Sets

Sets store unique items with average $O(1)$ membership tests. They are great for visited tracking and de-duplication. Use set operations for quick intersections or unions.

```
seen = set([1, 2, 2, 3])
print(seen)
print(2 in seen)
```

Output:

```
{1, 2, 3}
True
```

6) Tuples

Tuples are immutable and hashable, so they can be dict keys or set items. They are useful for grid coordinates and fixed pairs. Tuple unpacking makes code concise.

```
point = (2, 3)
x, y = point
print(x + y)
```

Output:

```
5
```

7) Slicing and Copying

Slicing creates shallow copies of lists and strings. Use `nums[:]` or `nums.copy()` to avoid aliasing bugs. For deep copy of nested lists, use `copy.deepcopy`.

```
nums = [1, 2, 3]
copy_nums = nums[:]
copy_nums.append(4)
print(nums, copy_nums)
```

Output:

```
[1, 2, 3] [1, 2, 3, 4]
```

8) Two Pointers

Two pointers reduce time from $O(n^2)$ to $O(n)$ for sorted or linear scans. Common for reversing, partitioning, or sliding windows. Use `left` and `right` indices that move inward.

```
nums = [1, 2, 3, 4]
left, right = 0, len(nums) - 1
while left < right:
    nums[left], nums[right] = nums[right], nums[left]
    left += 1
    right -= 1
print(nums)
```

Output:

```
[4, 3, 2, 1]
```

9) Sliding Window

Sliding windows handle subarray/substring constraints efficiently. Use a running window with **left** and **right**, updating counts as you move. Great for "longest" or "smallest" subarray problems.

```
s = "abca"
seen = set()
left = 0
best = 0
for right, ch in enumerate(s):
    while ch in seen:
        seen.remove(s[left])
        left += 1
    seen.add(ch)
    best = max(best, right - left + 1)
print(best)
```

Output:

3

10) Stack

Stacks are used for parentheses matching, monotonic stacks, and DFS. Python list works as a stack with **append** and **pop**. Always check for empty stack before popping.

```
stack = []
for ch in "(()())":
    if ch == "(":
        stack.append(ch)
    else:
        stack.pop()
print(len(stack) == 0)
```

Output:

True

11) Queue and Deque

Queues are FIFO; use **collections.deque** for O(1) pops from left. Deques are essential for BFS and sliding window problems. Avoid `list.pop(0)` because it is O(n).

```
from collections import deque
q = deque([1, 2, 3])
q.append(4)
print(q.popleft())
```

Output:

```
1
```

12) Heap (Priority Queue)

Heaps support quick access to the smallest item in $O(\log n)$ time. Python's `heapq` is a min-heap; use negatives for max-heap behavior. Common for k-th element and merge problems.

```
import heapq
heap = [3, 1, 2]
heapq.heapify(heap)
print(heapq.heappop(heap))
```

Output:

```
1
```

13) Sorting

Sorting is $O(n \log n)$ and often simplifies logic. Know stable sorting and the `key` parameter. Sorting tuples or lists of pairs is very common.

```
pairs = [(2, "b"), (1, "a"), (2, "a")]
pairs.sort(key=lambda x: (x[0], x[1]))
print(pairs)
```

Output:

```
[(1, 'a'), (2, 'a'), (2, 'b')]
```

14) Binary Search

Binary search applies to sorted data or monotonic predicates. It runs in $O(\log n)$ and is a core interview pattern. Python has `bisect` for insertion points.

```
import bisect
nums = [1, 3, 5, 7]
print(bisect.bisect_left(nums, 5))
```

Output:

```
2
```

15) Prefix Sums

Prefix sums allow $O(1)$ range sum queries after $O(n)$ preprocessing. Use them for subarray sum and frequency problems. Be careful about off-by-one indexing.

```
nums = [1, 2, 3]
pre = [0]
for n in nums:
    pre.append(pre[-1] + n)
print(pre)
print(pre[3] - pre[1])
```

Output:

```
[0, 1, 3, 6]
5
```

16) Sorting with Custom Order

Custom order solves problems like alien dictionary or custom ranking. Use a dict to map each char to its rank. Then sort using the rank as a key.

```
order = {"c": 0, "a": 1, "b": 2}
chars = ["a", "b", "c"]
chars.sort(key=lambda x: order[x])
print(chars)
```

Output:

```
['c', 'a', 'b']
```

17) Recursion Basics

Recursion needs a base case and a recursive step. Python has a recursion limit, so prefer iteration for deep recursion. Use memoization for repeated subproblems.

```
def factorial(n):  
    if n <= 1:  
        return 1  
    return n * factorial(n - 1)  
  
print(factorial(5))
```

Output:

```
120
```

18) DFS (Depth-First Search)

DFS explores a path fully before backtracking. Use recursion or an explicit stack. Great for tree/graph traversals and connected components.

```
graph = {0: [1, 2], 1: [3], 2: [], 3: []}  
visited = set()  
  
def dfs(node):  
    visited.add(node)  
    for nei in graph[node]:  
        if nei not in visited:  
            dfs(nei)  
  
dfs(0)  
print(visited)
```

Output:

```
{0, 1, 2, 3}
```

19) BFS (Breadth-First Search)

BFS explores level by level using a queue. It finds shortest paths in unweighted graphs. Use a deque for $O(1)$ pops from the left.

```
from collections import deque

graph = {0: [1, 2], 1: [3], 2: [], 3: []}
visited = set([0])
q = deque([0])

while q:
    node = q.popleft()
    for nei in graph[node]:
        if nei not in visited:
            visited.add(nei)
            q.append(nei)

print(visited)
```

Output:

```
{0, 1, 2, 3}
```

20) Trees: Traversals

Know preorder, inorder, and postorder patterns. They show up in BST problems and recursion practice. Iterative traversals are often required to avoid recursion depth issues.

```
class Node:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

root = Node(2, Node(1), Node(3))

res = []
stack = [root]
while stack:
    node = stack.pop()
    if not node:
        continue
    res.append(node.val)
    stack.append(node.right)
    stack.append(node.left)

print(res)
```

Output:


```
[2, 1, 3]
```

21) Graph Representation

Use adjacency lists for sparse graphs; they are memory efficient. Graphs can be directed or undirected; store both directions if undirected. Keep a visited set to avoid infinite loops.

```
edges = [(0, 1), (0, 2), (1, 2)]
adj = {0: [], 1: [], 2: []}
for a, b in edges:
    adj[a].append(b)
    adj[b].append(a)
print(adj)
```

Output:

```
{0: [1, 2], 1: [0, 2], 2: [0, 1]}
```

22) Topological Sort (Kahn's Algorithm)

Topological sort orders nodes in a DAG by dependency. Use in-degree counts and a queue of zero in-degree nodes. If not all nodes are processed, there is a cycle.

```
from collections import deque

edges = [(0, 1), (0, 2), (1, 3), (2, 3)]
n = 4
indeg = [0] * n
adj = {i: [] for i in range(n)}
for a, b in edges:
    adj[a].append(b)
    indeg[b] += 1

q = deque([i for i in range(n) if indeg[i] == 0])
order = []
while q:
    node = q.popleft()
    order.append(node)
    for nei in adj[node]:
        indeg[nei] -= 1
        if indeg[nei] == 0:
            q.append(nei)

print(order)
```

Output:

```
[0, 1, 2, 3]
```

23) Union-Find (Disjoint Set Union)

Union-Find tracks connected components with path compression. **find** returns the root; **union** merges two sets. It is common in graph connectivity and Kruskal's MST.

```
parent = [0, 1, 2, 3]
rank = [0, 0, 0, 0]

def find(x):
    if parent[x] != x:
        parent[x] = find(parent[x])
    return parent[x]

def union(a, b):
    ra, rb = find(a), find(b)
    if ra == rb:
        return
    if rank[ra] < rank[rb]:
        parent[ra] = rb
    elif rank[ra] > rank[rb]:
        parent[rb] = ra
    else:
        parent[rb] = ra
        rank[ra] += 1

union(0, 1)
union(2, 3)
print(find(1) == find(0), find(2) == find(3))
```

Output:

```
True True
```

24) Dynamic Programming Basics

DP stores results of overlapping subproblems to avoid recomputation. Use memoization (top-down) or tabulation (bottom-up). Always define the state, transition, and base cases clearly.

```
def fib(n):
    dp = [0, 1] + [0] * (n - 1)
    for i in range(2, n + 1):
```

```
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]

print(fib(7))
```

Output:

13

25) Greedy Strategy

Greedy picks the locally best option at each step. It works when the problem has the greedy-choice property. Use proof or counterexamples to validate greediness.

```
coins = [25, 10, 5, 1]
amount = 37
count = 0
for c in coins:
    count += amount // c
    amount %= c
print(count)
```

Output:

4

26) Backtracking

Backtracking explores choices, then undoes them to try alternatives. It is essential for permutations, combinations, and subset problems. Prune branches early to reduce runtime.

```
res = []
nums = [1, 2]

def backtrack(path, used):
    if len(path) == len(nums):
        res.append(path[:])
        return
    for i in range(len(nums)):
        if used[i]:
            continue
        used[i] = True
        path.append(nums[i])
        backtrack(path, used)
```

```
        path.pop()
        used[i] = False

    backtrack([], [False, False])
    print(res)
```

Output:

```
[[1, 2], [2, 1]]
```

27) Bit Manipulation Basics

Bit tricks can optimize speed and space for certain problems. Know AND, OR, XOR, shifts, and how to test bits. XOR is used for finding unique elements.

```
a = 5    # 101
b = 3    # 011
print(a & b)
print(a ^ b)
```

Output:

```
1
6
```

28) Math Utilities

Know `gcd`, `lcm`, and modular arithmetic. Use modulo to avoid overflow and stay within constraints. Python's `pow` supports fast modular exponentiation.

```
import math
print(math.gcd(12, 18))
print(pow(2, 10, 1000))
```

Output:

```
6
24
```

29) Complexity Awareness

Always reason about time and space complexity. Avoid nested loops when a hash map or two-pointer approach works. State complexity explicitly in your solution.

```
nums = [1, 2, 3, 4]
print(sum(nums))
```

Output:

```
10
```

30) Coding Style for Interviews

Write clear variable names and short helper functions. Handle edge cases early to simplify logic. Add comments only when the logic is not obvious.

```
nums = [1, 2, 3]
if not nums:
    print(0)
else:
    print(nums[-1])
```

Output:

```
3
```